CAT-3 COMPLETE NOTES

26CSE0105 · Structured Programming Concepts

Lectures 16–20 | Iteration, Nested Loops, Jump Statements,
Pattern Programming & Problem Solving

Prepared for revision: These notes follow the CAT-3 announcement and the two SPC lecture PDFs linked on StudyDesk (Lectures 16–18 and 19–20). They explain the concepts in simple language and include C examples, output patterns, dry-run guidance and common exam mistakes.

What is included

✓ while, do-while and for loops

✓ Nested loops and infinite loops

✓ break, continue and goto

✓ Star, number and alphabet patterns

✓ Problem solving using iteration

✓ Quick comparison tables and exam revision checklist

Syllabus map

Lecture	Topics from the CAT-3 announcement
16	Iterative statements: while, do-while and for loop
17–18	Nested loops, infinite loops, break, continue and goto statements
19–20	Pattern programming and problem solving using iteration

Study order: First understand how a single loop runs. Then learn nested loops. After that, practise jump statements, patterns and number-based problems. Most programming questions become easier when you can trace the values of the loop variables.

1. Introduction to iteration and loops

Iteration means repeating a set of instructions. A **loop** runs a block of code again and again while its condition allows it to continue. Loops save time and avoid writing the same statement many times.

Basic parts of a loop

• **Initialization:** gives the loop variable its starting value, e.g. `int i = 1;`

• **Condition:** decides whether another iteration should run, e.g. `i <= 5`

• **Body:** statements that repeat.

• **Updation:** changes the loop variable, e.g. `i++` or `i--`.

Iteration: one complete execution of the loop body is called one iteration.

Entry-controlled vs exit-controlled

Type	When is condition checked?	Possible number of executions
Entry-controlled	Before the loop body	0 or more times
Exit-controlled	After the loop body	At least 1 time

The `for` and `while` loops are entry-controlled. The `do-while` loop is exit-controlled.

2. The for loop

A `for` loop is useful when the number of repetitions is known or controlled by a counter.

Syntax

```
for (initialization; condition; update)
{
    // statements to repeat
}
```

Execution order: initialization happens once → condition is checked → body executes if true → update happens → condition is checked again. When the condition becomes false, the loop ends.

Example: print 1 to 5

```
#include <stdio.h>
int main(void)
{
    int i;
    for (i = 1; i <= 5; i++)
    {
        printf("%d ", i);
    }
    return 0;
}
```

```
// Output: 1 2 3 4 5
```

How to read a for loop

Part	Meaning in for (i = 1; i <= 5; i++)
i = 1	Start i at 1 (only once)
i <= 5	Continue while i is at most 5
i++	Increase i by 1 after each body execution

Common variations

Count backwards

```
for (int i = 5; i >= 1; i--)
    printf("%d ", i);
// Output: 5 4 3 2 1
```

Increase by 2

```
for (int i = 2; i <= 10; i += 2)
    printf("%d ", i);
// Output: 2 4 6 8 10
```

3. The while loop

A `while` loop checks its condition before executing the body. It is useful when the number of repetitions is not known in advance.

Syntax

```
initialization;
while (condition)
{
    // statements
    update;
}
```

Example: print 1 to 5

```
#include <stdio.h>
int main(void)
{
    int i = 1;
    while (i <= 5)
    {
        printf("%d ", i);
        i++;
    }
    return 0;
}
// Output: 1 2 3 4 5
```

Important: Do not forget to update the loop variable when the condition depends on it. If the condition never becomes false, the loop may run forever.

4. The do-while loop

A `do-while` loop executes the body first and checks the condition afterwards. Therefore, its body always runs at least once.

Syntax

```
initialization;
do
{
    // statements
    update;
} while (condition);
```

Example

```c
#include <stdio.h>
int main(void)
{
    int i = 1;
    do
    {
        printf("%d ", i);
        i++;
    } while (i <= 5);

    return 0;
}
// Output: 1 2 3 4 5
```

Syntax trap: A do-while loop ends with a semicolon after the condition: `while (condition);`

5. Difference between the three loops

Feature	for	while	do-while
Condition checked	Before body	Before body	After body
Minimum runs	0	0	1
Best suited for	Counter-controlled repetition	Condition-controlled repetition	At-least-once execution
Semicolon after condition?	No	No	Yes

Same task using different loops

All three can print the numbers 1 to 5. The choice depends on the problem and how clearly the loop can be written.

Equivalent while and do-while versions

```c
int i = 1;
while (i <= 5) {
    printf("%d ", i);
    i++;
}

int j = 1;
do {
    printf("%d ", j);
    j++;
} while (j <= 5);
```

6. Nested loops

A **nested loop** is a loop placed inside another loop. The outside loop is the **outer loop**; the loop inside it is the **inner loop**. For every one iteration of the outer loop, the inner loop completes all its iterations.

Example: outer loop 2 times, inner loop 3 times

```
for (int i = 1; i <= 2; i++)
{
    for (int j = 1; j <= 3; j++)
    {
        printf("i=%d j=%d\n", i, j);
    }
}
```

The inner body runs 2 × 3 = 6 times. The values are (1,1), (1,2), (1,3), then (2,1), (2,2), (2,3).

Types of nested loops

• Nested for: a for loop inside another for loop.

• Nested while: a while loop inside another while loop.

• Nested do-while: a do-while loop inside another do-while loop.

• Mixed nesting: one loop type can be placed inside another loop type.

Important reset rule

If the inner loop variable is declared outside the inner loop, reset it before starting each new inner-loop run. Otherwise, it may retain its old value and the inner loop may not execute as expected.

Correct nested while loop

```
int i = 1;
while (i <= 3)
{
    int j = 1;       // reset for every outer iteration
    while (j <= 2)
    {
        printf("* ");
        j++;
    }
    printf("\n");
    i++;
}
```

7. Infinite loops

An **infinite loop** is a loop that does not stop on its own because its condition never becomes false, or because it is intentionally written without an ending condition.

Common infinite-loop forms

```
while (1)
{
    // repeats unless control leaves the loop
}

for (;;)
{
    // repeats unless control leaves the loop
}
```

Common causes

• Forgetting to update the loop variable.

• Updating in the wrong direction, e.g. increasing a value when it must decrease to reach the stopping condition.

• Writing a condition that always remains true.

• Accidentally changing the wrong variable.

How to prevent accidental infinite loops

• Check that initialization gives a sensible starting value.

• Check that the condition can eventually become false.

• Check that the update moves the variable towards the stopping condition.

• Dry-run the first few iterations before running the program.

Example of a bug: `int i=1; while(i<=5) { printf("%d", i); }` never changes i, so the condition stays true. Add `i++;` inside the loop.

8. Jump statements

Jump statements change the normal flow of execution. The CAT-3 announcement includes `break`, `continue` and `goto`.

8.1 break

`break` immediately terminates the nearest enclosing loop or a switch statement. Execution continues with the first statement after that loop or switch.

Example

```
for (int i = 1; i <= 10; i++)
{
    if (i == 6)
        break;
    printf("%d ", i);
}
// Output: 1 2 3 4 5
```

8.2 continue

`continue` skips the remaining statements in the current iteration and moves to the next iteration of the loop.

Example: skip 2

```
for (int i = 1; i <= 5; i++)
{
    if (i == 2)
        continue;
    printf("%d ", i);
}
// Output: 1 3 4 5
```

8.3 goto

`goto` transfers control to a labelled statement within the same function. A label is an identifier followed by a colon. Excessive use of goto can make code difficult to follow, so structured loops are usually preferred.

Example

```
#include <stdio.h>
int main(void)
{
    int n = 4;

    if (n % 2 == 0)
        goto even;
    else
        goto odd;

even:
    printf("Even");
    return 0;

odd:
    printf("Odd");
    return 0;
}
```

break vs continue vs goto

Statement	Effect
break	Ends the nearest loop or switch immediately.
continue	Skips the rest of the current loop iteration.
goto	Jumps to a labelled statement in the same function.

Exam reminder: In nested loops, `break` and `continue` affect the nearest enclosing loop, not automatically every loop.

9. Pattern programming

Pattern programming uses loops to print shapes made of stars, numbers or letters. For a basic 2D pattern, the outer loop usually controls the **rows** and the inner loop controls what is printed in each **row**.

A simple method for solving any pattern

• Count the number of rows.

• Write down how many symbols appear in each row.

• Find the relationship between the row number and the symbol count.

• Use the outer loop for rows and inner loop(s) for spaces and symbols.

• Print a newline after completing each row.

Pattern 1: square of stars

`n = 4` produces:

Output

```
* * * *
* * * *
* * * *
* * * *
```

C code

```
for (int i = 1; i <= 4; i++)
{
    for (int j = 1; j <= 4; j++)
        printf("* ");
    printf("\n");
}
```

Pattern 2: right-angled triangle

Output

```
*
* *
* * *
* * * *
```

C code

```
for (int i = 1; i <= 4; i++)
{
    for (int j = 1; j <= i; j++)
        printf("* ");
    printf("\n");
}
```

The number of stars in row i is i.

Pattern 3: inverted right-angled triangle

Output

```
* * * *
* * *
* *
*
```

C code

```
for (int i = 4; i >= 1; i--)
{
    for (int j = 1; j <= i; j++)
        printf("* ");
    printf("\n");
}
```

Pattern 4: centred/equilateral pyramid

Output

```
    *
   * * *
  * * * * *
 * * * * * * *
```

C code

```
int rows = 4;
for (int i = 1; i <= rows; i++)
{
    for (int s = 1; s <= rows - i; s++)
        printf(" ");
    for (int j = 1; j <= 2*i - 1; j++)
        printf("*");
    printf("\n");
}
```

Spaces decrease by one in each row. Stars follow 1, 3, 5, 7… which is 2 × row – 1.

Pattern 5: hollow triangle

Output

```
    *
   * *
  *   *
 *******
```

C code

```
int rows = 4;
for (int i = 1; i <= rows; i++)
{
    for (int s = 1; s < rows - i + 1; s++)
        printf(" ");

    for (int j = 1; j <= 2*i - 1; j++)
    {
        if (i == rows || j == 1 || j == 2*i - 1)
            printf("*");
        else
            printf(" ");
    }
    printf("\n");
}
```

Pattern 6: increasing number triangle

Output

```
1
1 2
1 2 3
1 2 3 4
```

C code

```
for (int i = 1; i <= 4; i++)
{
    for (int j = 1; j <= i; j++)
        printf("%d ", j);
    printf("\n");
}
```

Pattern 7: inverted number triangle

Output

```
1 2 3 4
1 2 3
1 2
1
```

C code

```
for (int i = 4; i >= 1; i--)
{
    for (int j = 1; j <= i; j++)
        printf("%d ", j);
    printf("\n");
}
```

Pattern 8: increasing alphabet triangle

Output

```
A
A B
A B C
A B C D
```

C code

```
for (int i = 1; i <= 4; i++)
{
    char ch = 'A';
    for (int j = 1; j <= i; j++)
        printf("%c ", ch++);
    printf("\n");
}
```

Pattern 9: inverted alphabet triangle

Output

```
A B C D
A B C
A B
A
```

C code

```
for (int i = 4; i >= 1; i--)
{
    char ch = 'A';
    for (int j = 1; j <= i; j++)
        printf("%c ", ch++);
    printf("\n");
}
```

Pattern 10: diamond idea

A diamond is made by combining an increasing pyramid with a decreasing inverted pyramid. The first half increases the number of stars; the second half decreases it. Use spaces to centre both halves.

Simple filled diamond (n = 3)

```
   *
  ***
 *****
  ***
   *
```

10. Problem solving using iteration

Iteration can solve problems involving repeated calculations, digit processing, counting and checking number properties. The following are useful practice programs for the announced problem-solving portion.

10.1 Sum of numbers from 1 to N

C code

```
int n, sum = 0;
scanf("%d", &n);
for (int i = 1; i <= n; i++)
    sum += i;
printf("%d", sum);
```

For N = 5, sum = 1 + 2 + 3 + 4 + 5 = 15.

10.2 Factorial of a number

Factorial of n is the product of all positive integers from 1 to n. By definition, 0! = 1.

C code

```
int n;
long long fact = 1;
scanf("%d", &n);

if (n < 0)
    printf("Factorial is not defined");
else {
    for (int i = 1; i <= n; i++)
        fact *= i;
    printf("%lld", fact);
}
```

10.3 Fibonacci series

The sequence starts 0, 1. Each next term is the sum of the previous two terms.

Print first N terms

```
int n;
int a = 0, b = 1, next;
scanf("%d", &n);

for (int i = 1; i <= n; i++) {
    printf("%d ", a);
    next = a + b;
    a = b;
    b = next;
}
```

10.4 Reverse a number

C code

```
int n, rev = 0, digit;
scanf("%d", &n);

while (n != 0) {
    digit = n % 10;       // last digit
    rev = rev * 10 + digit;
    n = n / 10;           // remove last digit
}
printf("%d", rev);
```

10.5 Sum of digits

C code

```
int n, sum = 0;
scanf("%d", &n);

while (n != 0) {
    sum += n % 10;
    n /= 10;
}
```

```
printf("%d", sum);
```

10.6 Count digits

C code

```
int n, count = 0;
scanf("%d", &n);

if (n == 0)
    count = 1;
else {
    while (n != 0) {
        count++;
        n /= 10;
    }
}
printf("%d", count);
```

10.7 Palindrome number

A number is a palindrome if it reads the same forward and backward, e.g. 121 or 1331. Reverse the number and compare it with the original.

C code

```
int n, original, rev = 0, digit;
scanf("%d", &n);
original = n;

while (n > 0) {
    digit = n % 10;
    rev = rev * 10 + digit;
    n /= 10;
}

if (original == rev)
    printf("Palindrome");
else
    printf("Not palindrome");
```

10.8 Prime number

A prime number is an integer greater than 1 that has exactly two positive factors: 1 and itself. It is enough to test possible divisors from 2 while i × i is at most n.

C code

```
int n, prime = 1;
scanf("%d", &n);

if (n <= 1)
    prime = 0;
else {
    for (int i = 2; i * i <= n; i++) {
        if (n % i == 0) {
            prime = 0;
            break;
        }
    }
}

if (prime)
    printf("Prime");
else
    printf("Not prime");
```

10.9 Multiplication table

C code

```
int n;
scanf("%d", &n);
for (int i = 1; i <= 10; i++)
    printf("%d x %d = %d\n", n, i, n*i);
```

10.10 Even numbers up to N

C code

```
int n;
scanf("%d", &n);
for (int i = 2; i <= n; i += 2)
    printf("%d ", i);
```

11. Dry-running and tracing loops

A dry run means manually following the program line by line and recording variable values. It is especially useful for output-based MCQs.

Example trace

Program

```
int i = 1;
while (i <= 4) {
    printf("%d ", i);
    i += 2;
}
```

Iteration	i at condition	Condition	Printed	i after update
1	1	True	1	3
2	3	True	3	5
Stop	5	False	—	—

Output: 1 3

A reliable dry-run checklist

• Write the initial value of every loop variable.

• Check the condition before the body for for/while loops.

• Execute statements in the exact written order.

• Apply the update at the correct point.

• For nested loops, finish the inner loop before advancing the outer loop.

• For continue, skip the remaining body; for break, leave the nearest loop.

12. Common exam mistakes

• Using = instead of == in a comparison.

• Forgetting the semicolon at the end of a do-while statement.

• Putting an extra semicolon after a for or while header.

• Forgetting to update a counter, causing an infinite loop.

• Using `i < n` when the question requires i to reach n.

• Confusing `break` (exit loop) with `continue` (skip current iteration).

• Not resetting the inner loop counter in a nested while loop.

• Printing a newline in the wrong place in a pattern.

• Using the wrong loop boundary, causing one extra or one fewer row.

13. Quick revision sheet

Concept	Remember
for	Initialization; condition; update. Entry-controlled.
while	Checks condition first. May execute zero times.
do-while	Executes body first. Runs at least once; ends with semicolon.
Nested loop	Inner loop completes for each outer-loop iteration.
Infinite loop	The stopping condition never becomes false.
break	Immediately exits nearest loop or switch.
continue	Skips the rest of the current iteration.
goto	Jumps to a label in the same function.
Pattern	Outer loop = rows; inner loop(s) = spaces/symbols.
Digit extraction	n % 10 gets last digit; n / 10 removes last digit.

14. CAT-3 preparation checklist

Use this list to check whether you are ready. Try writing each program without looking at the notes.

■ I can explain the difference between for, while and do-while.

■ I can predict the output of a loop by dry-running it.

■ I can identify and correct an accidental infinite loop.

■ I understand nested loops and can count total inner-loop executions.

■ I know the effect of break, continue and goto.

■ I can write square, triangle, inverted triangle and pyramid patterns.

■ I can write number and alphabet patterns.

■ I can solve factorial, Fibonacci, reverse, digit sum and palindrome problems.

■ I can check whether a number is prime.

■ I can identify common syntax and boundary-condition mistakes.

Source alignment: The lecture-specific explanations and pattern examples are based on StudyDesk's SPC PDFs for Lectures 16–18 (Iterative Statements) and Lectures 19–20 (Pattern Programming). The CAT-3 announcement also names infinite loops and problem solving using iteration, so those topics are included as additional syllabus coverage.

End of CAT-3 SPC Notes